

Coding For Placement Series

C Track

CDP 101 to 118

PREREQUISITE HANDOUT

Self-Study Material

BIZOTIC

Empowering students towards their first job!

v1

1. Introduction

C is the language you learnt in your first year. It is also the language a large share of campus coding tests are still written and evaluated in. TCS NQT, Infosys, Cognizant, Capgemini and Accenture all accept C in their coding rounds, and product companies continue to use C for operating systems, embedded controllers, device drivers, databases and network stacks. Nothing you learn here becomes obsolete. C is the language that explains how every other language works underneath.

This handout is not a first course in C. It assumes you have already studied it in your first year. What it assumes you do **not** have is a compiler on your laptop and the syntax still at your fingertips, because a long gap has passed since you last wrote and ran a C program, and most of it has gone quiet.

That gap is the problem this document solves. The classroom program starts at problem solving on day one. Theory time is deliberately short. If you walk in still trying to remember whether the semicolon goes after the closing brace of a `do-while`, you will spend the first two days catching up instead of solving problems, and those two days are not recoverable.

Work through this document completely before day one of the program. Install the compiler, run every example yourself rather than reading it, take the diagnostic in Section 6 honestly, and attempt all twenty practice problems in Section 15. Budget six to eight hours across the week. Students who arrive having done this start the program at full speed.

---

2. Tools You Need

You need a C compiler and a text editor. The standard for this program is **gcc version 13 or later**, used from a terminal, because that is exactly how the classroom sessions run.

Windows

Two installs, in this order.

Install 1: MSYS2, which provides the gcc compiler.

1. Download the installer from `https://www.msys2.org` and run it. Accept the default install location `C:\msys64`.
2. When the installation finishes, open the application named **MSYS2 UCRT64** from the Start menu.
3. At that prompt, type the following and press Enter. Accept the defaults when it asks.

```
pacman -S --needed base-devel mingw-w64-ucrt-x86_64-toolchain
```

4. Add the compiler to your system PATH so it works from any terminal. Open Start, search for "Edit the system environment variables", click **Environment Variables**, select **Path** under User variables, click **Edit**, click **New**, and add:

```
C:\msys64\ucrt64\bin
```

5. Click OK on every window. Close any terminal that was already open.

Install 2: Visual Studio Code, which is your editor. Download it from `https://code.visualstudio.com/download` and install with default options. Install the extension named **C/C++** by Microsoft from the Extensions panel.

macOS

Open Terminal and run:

```
xcode-select --install
```

Linux (Ubuntu or Debian)

Open a terminal and run:

```
sudo apt update
sudo apt install build-essential
```



Verify your installation

Open a fresh terminal. On Windows, use Command Prompt or the terminal inside VS Code. Type:

```
gcc --version
```

You should see a version number of 13 or higher. If you see "not recognised" or "command not found", the install did not complete or the PATH entry is wrong. Repeat the step above before day one, not on day one.

If the install will not work

Use `https://www.onlinegdb.com` with the language set to C. This is a genuine fallback and you can complete every example and practice problem in this handout on it. It is not a substitute for a working local compiler, so keep trying the install, but never let a failed install be the reason you arrive unable to write code.

---

3. Your First C Program

Create a file named `hello.c` and type this exactly.

```
#include <stdio.h>

int main(void) {
    printf("Hello, World\n");
    return 0;
}
```

Compile it and run it. From the terminal, in the folder where you saved the file:

```
gcc hello.c -o hello
./hello
```

On Windows Command Prompt the second line is `hello` or `hello.exe` instead of `./hello`.

Output:

```
Hello, World
```

What each line does.

Line	Meaning
`#include <stdio.h>`	Pulls in the standard input and output library. Without it, `printf` and `scanf` do not exist.
`int main(void)`	The entry point. Execution starts here. `int` means this function hands an integer back to the operating system.
`printf("Hello, World\n");`	Prints text. `\n` is an escape sequence meaning "move to the next line". It is not two characters on screen, it is one newline.
`return 0;`	Tells the operating system the program finished cleanly. Any non-zero value signals failure.

C is compiled, not interpreted. `gcc` translates your source file into a separate executable file before anything runs. If compilation fails, no executable is produced and nothing runs at all. This is different from Python, where the program starts running and fails partway. In C, a syntax error anywhere in the file means the whole program does not exist yet.

Common escape sequences:

Sequence	Effect
`\n`	New line
`\t`	Tab
`\\`	A single backslash
`\"`	A double quote inside a string



Sequence	Effect
`%%`	A literal percent sign inside `printf`

4. Four Things That Will Trip You Up

Some C material still in circulation, including older textbooks and tutorials, uses forms that a modern compiler will not accept. If your program fails to compile with one of the messages below, this is why. Four items, and the fix for each.

`void main()` is wrong. Use `int main(void)`.

```
#include <stdio.h>
void main() {
    printf("Hello");
}
```

gcc says:

```
warning: return type of 'main' is not 'int' [-Wmain]
    2 | void main() {
      |      ^~~~
```

It still runs, but it is not standard C, and some online judges reject it outright. Write `int main(void)` and end with `return 0;`. Always.

`conio.h`, `clrscr()` and `getch()` do not exist.

```
#include <stdio.h>
#include <conio.h>

void main() {
    clrscr();
    printf("Hello");
    getch();
}
```

gcc says:

```
fatal error: conio.h: No such file or directory
    2 | #include <conio.h>
      |          ^~~~~~~~
compilation terminated.
```

`conio.h` is not part of standard C. It was an extension shipped with some older compilers and it does not exist on gcc, so nothing that depends on it will build. Delete every line that uses it. You also do not need `getch()` to hold the screen open, because you are running from a terminal and the terminal stays open.

`int` is 4 bytes, and the ceiling is 2147483647.

Check the sizes on your own machine rather than trusting a number you have seen quoted. If you have come across a limit of 32,767 anywhere, that belongs to a 2 byte `int` and does not apply on gcc.

```
#include <stdio.h>
#include <limits.h>

int main(void) {
    printf("sizeof(int)    = %zu bytes\n", sizeof(int));
    printf("sizeof(float)  = %zu bytes\n", sizeof(float));
    printf("sizeof(double) = %zu bytes\n", sizeof(double));
    printf("sizeof(char)   = %zu bytes\n", sizeof(char));
    printf("INT_MAX        = %d\n", INT_MAX);
    printf("INT_MIN        = %d\n", INT_MIN);
    return 0;
```

```
}
```

Output:

```
sizeof(int)    = 4 bytes
sizeof(float)  = 4 bytes
sizeof(double) = 8 bytes
sizeof(char)   = 1 bytes
INT_MAX        = 2147483647
INT_MIN        = -2147483648
```

Memorise 2147483647. It is the ceiling that breaks answers in coding tests, and Section 7 shows what happens when you cross it.

`gets()` is banned.

`gets()` was removed from the C standard because it cannot be used safely. Never write it. Line input is taught in class.

---

5. Understanding Errors

There are two kinds of failure and they happen at different moments. Knowing which one you are looking at is half of debugging.

Compile error: the program was never built

```
#include <stdio.h>

int main(void) {
    int marks = 40
    printf("%d\n", marks);
    return 0;
}
error: expected ',' or ';' before 'printf'
    5 |     printf("%d\n", marks);
      |     ^~~~~~
```

How to read it. The number `5` is the line gcc was reading when it gave up. The actual mistake is on line 4, the missing semicolon. This is the single most important habit to build: **a compile error frequently points at the line after the real mistake.** Always check the line above the one named.

Another one you will meet constantly:

```
#include <stdio.h>

int main(void) {
    total = 10;
    printf("%d\n", total);
    return 0;
}
error: 'total' undeclared (first use in this function)
    4 |     total = 10;
      |     ^~~~~
```

In C every variable must be declared with a type before use. There is no automatic creation.

Warning: the program was built, but it is probably wrong

```
#include <stdio.h>

int main(void) {
    float cgpa = 8.25f;
    printf("%d\n", cgpa);
    return 0;
}
warning: format '%d' expects argument of type 'int', but argument 2 has type
'double' [-Wformat=]
```



This compiles and runs, and prints a large meaningless number that differs from machine to machine. C does not check that your format specifier matches your argument. **Treat every warning as an error.** Compile with warnings switched on and fix all of them:

```
gcc -Wall -Wextra hello.c -o hello
```

Runtime error: the program was built and then crashed

```
#include <stdio.h>

int main(void) {
    int total = 250;
    int students = 0;

    printf("Average = %d\n", total / students);
    return 0;
}
```

This compiles without complaint. When you run it:

```
Floating point exception
```

Dividing an integer by zero is a crash, not a warning. The compiler cannot catch it because `students` only becomes zero while the program is running.

The three-way summary: a compile error means nothing ran. A warning means it ran but you should not trust it. A runtime crash means it ran and then died.

---

6. Ten Minute Diagnostic

Read each snippet and write down what you think it prints **before** you run it. Then run it. Every one of these is a mistake that costs marks in a real coding test. Answers and explanations follow immediately after, so cover them.

These are fragments, not whole programs. To run one, paste it inside the standard template from Section 3, between `int main(void) {` and `return 0;`. Snippet 10 additionally needs `#include <string.h>` at the top.

Snippet 1

```
printf("%d\n", 7 / 2);
printf("%d\n", 7 % 2);
printf("%.2f\n", 7.0 / 2);
```

Snippet 2

```
int a = 5;
int b = a++;
int c = ++a;
printf("a=%d b=%d c=%d\n", a, b, c);
```

Snippet 3

```
int i;
for (i = 0; i < 3; i++);
    printf("i = %d\n", i);
```

Snippet 4

```
int x = 10;
if (x = 0) {
    printf("zero\n");
} else {
    printf("not zero\n");
}
printf("x is now %d\n", x);
```

Snippet 5

```
int code = 2;
switch (code) {
    case 1: printf("One\n");
```

```
    case 2: printf("Two\n");
    case 3: printf("Three\n");
    default: printf("Other\n");
}
```

Snippet 6

```
char ch = 'A';
printf("%d\n", ch);
printf("%c\n", ch + 2);
printf("%d\n", 'z' - 'a');
```

Snippet 7

```
printf("%d\n", -7 / 2);
printf("%d\n", -7 % 3);
```

Snippet 8

```
int n = 0;
do {
    printf("body ran, n = %d\n", n);
    n++;
} while (n < 0);
```

Snippet 9

```
int i, j, count = 0;
for (i = 1; i <= 3; i++) {
    for (j = 1; j <= i; j++) {
        printf("*");
        count++;
    }
    printf("\n");
}
printf("count = %d\n", count);
```

Snippet 10

```
char city[] = "MYSURU";
printf("sizeof = %zu\n", sizeof(city));
printf("strlen = %zu\n", strlen(city));
printf("last char = %c\n", city[strlen(city) - 1]);
```

Answers

#	Output	What it tests
1	`3` then `1` then `3.50`	Integer division truncates. Add a decimal point to force real division.
2	`a=7 b=5 c=7`	`a++` hands over the old value then increments. `++a` increments first.
3	`i = 3`	The stray semicolon ends the loop body. The `printf` runs once, after the loop, with `i` already at 3.
4	`not zero` then `x is now 0`	`=` assigns, `==` compares. `x = 0` sets x to 0 and the whole expression evaluates to 0, which is false.
5	`Two Three Other`	A `case` without `break` falls through into every case below it.
6	`65` then `C` then `25`	A `char` is a small integer. `'A'` is 65, and arithmetic on it is ordinary arithmetic.
7	`-3` then `-1`	Integer division truncates toward zero, and `%` keeps the sign of the left operand. `-7 % 3` is not 2.
8	`body ran, n = 0`	A `do-while` always executes its body once, even when the condition is false from the start.
9	`* ** ***` then `count = 6`	The inner loop runs fully for each single outer iteration.
10	`sizeof = 7`, `strlen = 6`, `last char = U`	A string carries an invisible `'\0'` terminator. `sizeof` counts it, `strlen` does not.

Score yourself out of 10. Seven or above means your C is in reasonable shape and the rest of this document is revision. Below seven means work through every section that follows properly, running the code rather than reading it.

---

7. Variables and Data Types

A variable is a named box in memory that holds one value of one declared type. In C you must state the type before you use the name, and the type never changes afterwards.

```
int roll;              /* declaration only, holds garbage until assigned */
int marks = 78;        /* declaration with initialisation */
```

The four types you need

Type	Size	Holds	Format specifier
`int`	4 bytes	Whole numbers, -2147483648 to 2147483647	`%d`
`float`	4 bytes	Decimals, about 6 significant digits	`%f`
`double`	8 bytes	Decimals, about 15 significant digits	`%lf` in `scanf`, `%f` in `printf`
`char`	1 byte	One character, stored as its numeric code	`%c`

Use `int` by default. Use `double` rather than `float` for any calculation involving money, rates or averages, because `float` loses precision quickly.

Declaring, initialising and printing

```
#include <stdio.h>

int main(void) {
    int roll = 47;
    float cgpa = 8.25f;
    double distance = 1234.5678;
    char grade = 'A';

    printf("Roll number : %d\n", roll);
    printf("CGPA        : %.2f\n", cgpa);
    printf("Distance    : %.4lf\n", distance);
    printf("Grade       : %c\n", grade);
    return 0;
}
```

Output:

```
Roll number : 47
CGPA        : 8.25
Distance    : 1234.5678
Grade       : A
```

`%.2f` means "print with exactly two digits after the decimal point". `%.4lf` means four. You will use this constantly, because almost every coding test question that involves money asks for two decimals.

Naming rules

- Letters, digits and underscore only. No spaces, no hyphens.
- Cannot begin with a digit. `2total` is invalid, `total2` is fine.
- Case sensitive. `marks`, `Marks` and `MARKS` are three different variables.
- Cannot be a C keyword. `int`, `for`, `return`, `switch` and the rest are reserved.



- Use meaningful names. `totalMarks` beats `tm` beats `x`.

Always initialise

```
#include <stdio.h>

int main(void) {
    int counted = 0;
    counted = counted + 5;
    printf("counted = %d\n", counted);
    return 0;
}
```

Output:

```
counted = 5
```

Had `counted` been declared without `= 0`, it would have started with whatever was already in that memory, and the answer would be unpredictable. C does not zero your variables for you. Every accumulator starts at 0, every product accumulator starts at 1.

Trap: silent integer overflow

```
#include <stdio.h>
#include <limits.h>

int main(void) {
    volatile int big = INT_MAX;
    printf("INT_MAX        = %d\n", big);
    printf("INT_MAX plus 1 = %d\n", big + 1);
    return 0;
}
```

Output:

```
INT_MAX        = 2147483647
INT_MAX plus 1 = -2147483648
```

There is no error and no warning. The value silently wraps to the most negative number. If a coding test says the answer can be as large as ten to the power eighteen, `int` will not hold it and you must use `long long` with `%lld`. Factorials past 12 and repeated multiplications are where this bites most often.

---

8. Operators

Arithmetic

```
#include <stdio.h>

int main(void) {
    int a = 17, b = 5;
    printf("a + b = %d\n", a + b);
    printf("a - b = %d\n", a - b);
    printf("a * b = %d\n", a * b);
    printf("a / b = %d\n", a / b);
    printf("a %% b = %d\n", a % b);
    return 0;
}
```

Output:

```
a + b = 22
a - b = 12
a * b = 85
a / b = 3
a % b = 2
```

`%` is the remainder operator and it works only on integers. `17 % 5` is 2. `%` on a `float` is a compile error.



Trap 1: integer division

This is the most common wrong answer in C coding tests. When both operands are integers, C performs integer division and discards the fractional part. It does not round. It truncates.

```
#include <stdio.h>

int main(void) {
    int total = 7, parts = 2;
    printf("wrong : %f\n", (float)(total / parts));
    printf("right : %f\n", (float)total / parts);
    printf("right : %f\n", total / 2.0);
    return 0;
}
```

Output:

```
wrong : 3.000000
right : 3.500000
right : 3.500000
```

The first line is wrong because the division happened in integers first and the cast came too late. **Cast before dividing, or make one operand a decimal literal.**

Trap 2: negative operands

```
printf("%d\n", -7 / 2);   /* -3, truncates toward zero, not -4 */
printf("%d\n", -7 % 3);   /* -1, the sign follows the left operand */
```

If a problem requires a non-negative remainder, take the absolute value first.

Relational and logical

```
#include <stdio.h>

int main(void) {
    int age = 20, income = 18000;
    printf("age > 18            : %d\n", age > 18);
    printf("age == 20           : %d\n", age == 20);
    printf("age != 20           : %d\n", age != 20);
    printf("age > 18 && income < 25000 : %d\n", age > 18 && income < 25000);
    printf("age > 60 || income < 25000 : %d\n", age > 60 || income < 25000);
    printf("!(age > 18)         : %d\n", !(age > 18));
    return 0;
}
```

Output:

```
age > 18            : 1
age == 20           : 1
age != 20           : 0
age > 18 && income < 25000 : 1
age > 60 || income < 25000 : 1
!(age > 18)         : 0
```

C has no separate boolean type in the style you need here. **Zero is false. Every non-zero value is true, including negative numbers.** A comparison produces 1 for true and 0 for false.

`&&` is AND, `||` is OR, `!` is NOT. Note that `&&` and `||` short circuit: in `a && b`, if `a` is false, `b` is never evaluated at all.

Increment and decrement

```
#include <stdio.h>

int main(void) {
    int n = 10;
    int result;

    result = n++;
    printf("postfix: result = %d, n = %d\n", result, n);
```



```
    n = 10;
    result = ++n;
    printf("prefix : result = %d, n = %d\n", result, n);
    return 0;
}
```

Output:

```
postfix: result = 10, n = 11
prefix : result = 11, n = 11
```

`n++` hands over the old value and then increments. `++n` increments and then hands over the new value. When the result is not being used, as in a `for` loop update, the two are identical.

Never write both a read and an increment of the same variable in one statement, as in `printf("%d %d", n++, n)`. The result is undefined and different compilers give different answers.

Compound assignment

```
#include <stdio.h>

int main(void) {
    int total = 100;
    total += 25;  printf("after += 25 : %d\n", total);
    total -= 40;  printf("after -= 40 : %d\n", total);
    total *= 2;   printf("after *= 2  : %d\n", total);
    total /= 7;   printf("after /= 7  : %d\n", total);
    total %= 4;   printf("after %%= 4  : %d\n", total);
    return 0;
}
```

Output:

```
after += 25 : 125
after -= 40 : 85
after *= 2  : 170
after /= 7  : 24
after %= 4  : 0
```

`total += 25` is shorthand for `total = total + 25`.

Precedence

```
#include <stdio.h>

int main(void) {
    printf("2 + 3 * 4       = %d\n", 2 + 3 * 4);
    printf("(2 + 3) * 4     = %d\n", (2 + 3) * 4);
    printf("10 - 4 - 3      = %d\n", 10 - 4 - 3);
    printf("20 / 4 * 2      = %d\n", 20 / 4 * 2);
    printf("1 + 2 > 2 && 3 < 4 = %d\n", 1 + 2 > 2 && 3 < 4);
    return 0;
}
```

Output:

```
2 + 3 * 4       = 14
(2 + 3) * 4     = 20
10 - 4 - 3      = 3
20 / 4 * 2      = 10
1 + 2 > 2 && 3 < 4 = 1
```

Order, highest first: `!` and unary minus, then `* / %`, then `+ -`, then `< <= > >=`, then `== !=`, then `&&`, then `||`, then `=`. Operators of equal precedence group left to right, which is why `10 - 4 - 3` is 3 and not 9.

Do not memorise the full table. **Use brackets.** Brackets cost nothing and remove the entire category of mistake.

9. Input From the Keyboard

`scanf` reads typed values into variables. It needs two things: a format specifier that matches the type, and an ampersand before the variable name.

```
#include <stdio.h>

int main(void) {
    int roll;
    float cgpa;

    printf("Enter roll number: ");
    scanf("%d", &roll);
    printf("Enter CGPA: ");
    scanf("%f", &cgpa);

    printf("Roll %d has CGPA %.2f\n", roll, cgpa);
    return 0;
}
```

Input:

```
47
8.25
```

Output:

```
Enter roll number: Enter CGPA: Roll 47 has CGPA 8.25
```

About the ampersand. `&` means "the address of". For now, treat `scanf` as a pattern you type. The block on pointers is where the ampersand finally makes sense. Do not go looking for a deeper explanation before then, and do not forget it, because leaving it out is a crash rather than a wrong answer.

Specifiers in scanf

Type	scanf	printf
`int`	`%d`	`%d`
`float`	`%f`	`%f`
`double`	`%lf`	`%f`
`char`	`%c` (see the trap below)	`%c`

Note the asymmetry on `double`: `scanf` needs `%lf`, `printf` accepts `%f`. Getting this wrong is a silent garbage value, not an error.

Reading several values at once

```
#include <stdio.h>

int main(void) {
    int day, month, year;
    printf("Enter date as dd mm yyyy: ");
    scanf("%d %d %d", &day, &month, &year);
    printf("Date entered: %02d-%02d-%04d\n", day, month, year);
    return 0;
}
```

Input:

```
5 9 2026
```

Output:

```
Enter date as dd mm yyyy: Date entered: 05-09-2026
```

`%02d` pads with a leading zero to a width of two.

Trap: the leftover newline before `%c`



This one catches almost everybody. When you type `47` and press Enter, `scanf("%d", ...)` consumes the `47` and leaves the Enter key, the newline character, sitting in the input buffer. The next `%c` then reads that newline instead of waiting for you.

```
#include <stdio.h>

int main(void) {
    int roll;
    char grade;

    printf("Enter roll number: ");
    scanf("%d", &roll);
    printf("Enter grade letter: ");
    scanf("%c", &grade);

    printf("Roll %d, grade [%c]\n", roll, grade);
    return 0;
}
```

Input:

```
47
A
```

Output:

```
Enter roll number: Enter grade letter: Roll 47, grade [
]
```

The grade printed as a line break, because that is literally what was read. The `A` was never reached.

The fix is one space inside the format string, before the `%c`.

```
    scanf(" %c", &grade);
```

Input:

```
47
A
```

Output:

```
Enter roll number: Enter grade letter: Roll 47, grade [A]
```

A space in a `scanf` format string means "skip any amount of whitespace first". `%d` and `%f` skip leading whitespace automatically. `%c` does not. **Whenever a `%c` follows any other `scanf`, write `" %c"`.**

---

10. Type Casting

Conversion between types happens in two ways: implicitly, where C does it for you, and explicitly, where you demand it.

Implicit promotion happens when you mix types in one expression. C promotes the narrower type to the wider one, so `int` becomes `double` when combined with a `double`. This is why `total / 2.0` gives a correct decimal answer while `total / 2` does not.

Explicit casting is written as the target type in brackets in front of the value.

```
#include <stdio.h>

int main(void) {
    int marks = 287, subjects = 3;
    printf("int division      : %d\n", marks / subjects);
    printf("explicit cast     : %.2f\n", (float)marks / subjects);
    float price = 99.87f;
    printf("float to int      : %d\n", (int)price);
    char letter = 'K';
    printf("char to int       : %d\n", (int)letter);
    int code = 75;
    printf("int to char       : %c\n", (char)code);
```



```
    return 0;
}
```

Output:

```
int division      : 95
explicit cast     : 95.67
float to int      : 99
char to int       : 75
int to char       : K
```

Three things to take from this.

1. **Cast before the operation, never after.** `(float)marks / subjects` is right. `(float)(marks / subjects)` is wrong, because the integer division already happened inside the brackets.
2. **Casting a decimal to `int` truncates, it does not round.** 99.87 becomes 99, and 99.99 also becomes 99. To round a positive value to the nearest whole number, cast `value + 0.5`.
3. **A `char` and its numeric code are interchangeable.** `'K'` is 75. This is what lets you do `ch - 'a'` to get a letter's position in the alphabet, or `digit + '0'` to turn the number 7 into the character `'7'`.

Casting only changes how a value is read in that one expression. It does not change the declared type of the variable.

---

11. Conditionals

if, if-else and the else-if ladder

```
#include <stdio.h>

int main(void) {
    int marks = 72;

    if (marks >= 90) {
        printf("Grade S\n");
    } else if (marks >= 80) {
        printf("Grade A\n");
    } else if (marks >= 70) {
        printf("Grade B\n");
    } else if (marks >= 40) {
        printf("Grade C\n");
    } else {
        printf("Fail\n");
    }
    return 0;
}
```

Output:

```
Grade B
```

Order matters, because the first matching branch wins and the rest are skipped. Put the tightest condition first. Reverse the ladder and it silently produces the wrong grade for everybody:

```
#include <stdio.h>

int main(void) {
    int marks = 95;

    if (marks >= 40) {
        printf("wrong ladder says: Grade C\n");
    } else if (marks >= 90) {
        printf("wrong ladder says: Grade S\n");
    }
```

```
    return 0;
}
```

Output:

```
wrong ladder says: Grade C
```

No error, no warning, wrong answer. This is a very common way to lose marks in a coding test.

Nested if

```
#include <stdio.h>

int main(void) {
    int attendance = 82;
    int internal = 34;

    if (attendance >= 75) {
        if (internal >= 25) {
            printf("Eligible for exam\n");
        } else {
            printf("Blocked: internal marks short\n");
        }
    } else {
        printf("Blocked: attendance short\n");
    }
    return 0;
}
```

Output:

```
Eligible for exam
```

Use nesting when you need to report **which** condition failed. Use `&&` when you only need to know whether all of them passed. Those are different problems and they need different code shapes.

switch-case

```
#include <stdio.h>

int main(void) {
    int plan = 2;

    switch (plan) {
        case 1:
            printf("Veg plan, Rs 110 per day\n");
            break;
        case 2:
            printf("Non-veg plan, Rs 145 per day\n");
            break;
        case 3:
            printf("Special diet, Rs 170 per day\n");
            break;
        default:
            printf("Invalid plan code\n");
    }
    return 0;
}
```

Output:

```
Non-veg plan, Rs 145 per day
```

`switch` works on an exact match against an integer or a `char`. It cannot test a range and it cannot test a `float`. For ranges, use the ladder.

Every `case` needs its `break`. Without it, execution falls through into the next case and keeps going, as Snippet 5 in the diagnostic showed. Deliberate fall-through is occasionally useful for grouping cases, but an accidental one is a bug.

The ternary operator



```
#include <stdio.h>

int main(void) {
    int a = 37, b = 91;
    int larger = (a > b) ? a : b;
    printf("larger = %d\n", larger);
    printf("%d is %s\n", a, (a % 2 == 0) ? "even" : "odd");
    return 0;
}
```

Output:

```
larger = 91
37 is odd
```

`condition ? valueIfTrue : valueIfFalse`. Use it for a single short choice. Do not nest ternaries, because the result is unreadable.

Trap: = versus ==

`if (x = 5)` compiles cleanly, assigns 5 to `x`, and is always true because 5 is non-zero. `if (x = 0)` is always false. This is the single most dangerous typo in C. It is the reason Snippet 4 printed `not zero`. Read every condition back and check you wrote two equals signs.

Trap: dangling else

```
#include <stdio.h>

int main(void) {
    int attendance = 60;
    int internal = 30;

    if (attendance >= 75)
        if (internal >= 25)
            printf("Eligible\n");
    else
        printf("Attendance short\n");

    printf("checked\n");
    return 0;
}
```

Output:

```
checked
```

The indentation says the `else` belongs to the outer `if`, so a reader expects `Attendance short`. It does not print. **An `else` binds to the nearest unmatched `if`**, which is the inner one, and since attendance is 60 neither branch is reached at all. gcc even warns about it:

```
warning: suggest explicit braces to avoid ambiguous 'else' [-Wdangling-else]
```

House rule from here on: braces on every branch, always, even for a single statement. This removes the trap permanently and it is the style expected in the program.

---

12. Loops

The three loop forms

```
#include <stdio.h>

int main(void) {
    int i;

    printf("for   : ");
    for (i = 1; i <= 5; i++) printf("%d ", i);
    printf("\n");
```



```
    printf("while : ");
    i = 1;
    while (i <= 5) { printf("%d ", i); i++; }
    printf("\n");

    printf("do    : ");
    i = 1;
    do { printf("%d ", i); i++; } while (i <= 5);
    printf("\n");
    return 0;
}
```

Output:

```
for   : 1 2 3 4 5
while : 1 2 3 4 5
do    : 1 2 3 4 5
```

Which to use. `for` when you know the count in advance. `while` when you are looping until a condition changes, such as reading until a sentinel value. `do-while` when the body must run at least once, typically a menu or an input validation prompt.

`for (initialisation; condition; update)`. All three parts are optional. `for (;;)` is an infinite loop.

The `do-while` needs a semicolon after the closing `while (...)`. Forgetting it is a compile error.

Counting up, down, stepping, and accumulating

```
#include <stdio.h>

int main(void) {
    int i;

    printf("count down : ");
    for (i = 5; i >= 1; i--) printf("%d ", i);
    printf("\n");

    printf("step of 2  : ");
    for (i = 0; i <= 10; i += 2) printf("%d ", i);
    printf("\n");

    int sum = 0;
    for (i = 1; i <= 10; i++) sum += i;
    printf("sum 1 to 10 = %d\n", sum);

    int product = 1;
    for (i = 1; i <= 5; i++) product *= i;
    printf("5 factorial = %d\n", product);
    return 0;
}
```

Output:

```
count down : 5 4 3 2 1
step of 2  : 0 2 4 6 8 10
sum 1 to 10 = 55
5 factorial = 120
```

The accumulator pattern, which you will use in almost every coding test question: declare and initialise before the loop, update inside the loop, use after the loop. Write this on the inside of your eyelids: **a sum starts at 0, a product starts at 1, and a maximum starts at the first value, never at 0.** A maximum initialised to 0 gives the wrong answer the moment every input is negative.

Nested loops

```
#include <stdio.h>
```



```
int main(void) {
    int row, col;
    for (row = 1; row <= 4; row++) {
        for (col = 1; col <= row; col++) {
            printf("%d ", col);
        }
        printf("\n");
    }
    return 0;
}
```

Output:

```
1
1 2
1 2 3
1 2 3 4
```

The inner loop runs to completion for every single iteration of the outer loop. For patterns the mental model is fixed: **the outer loop is the row, the inner loop is the column, and what you print depends on the relationship between the two counters.** The `printf("\n")` belongs to the outer loop, after the inner loop finishes.

break and continue

```
#include <stdio.h>

int main(void) {
    int i;

    printf("continue skips 3 : ");
    for (i = 1; i <= 6; i++) {
        if (i == 3) continue;
        printf("%d ", i);
    }
    printf("\n");

    printf("break stops at 3 : ");
    for (i = 1; i <= 6; i++) {
        if (i == 3) break;
        printf("%d ", i);
    }
    printf("\n");
    return 0;
}
```

Output:

```
continue skips 3 : 1 2 4 5 6
break stops at 3 : 1 2
```

`continue` abandons the current iteration and jumps to the update step. `break` leaves the loop entirely. In nested loops, `break` exits **only the innermost loop**, not both.

Trap: the stray semicolon

```
#include <stdio.h>

int main(void) {
    int i;
    for (i = 0; i < 3; i++);
        printf("i = %d\n", i);
    return 0;
}
```

Output:

```
i = 3
```



The semicolon after the closing bracket is a complete, empty loop body. The loop spins three times doing nothing, then the `printf` runs once, with `i` already at 3. It compiles, so you will not be told. gcc gives a hint if you have warnings on:

```
warning: this 'for' clause does not guard... [-Wmisleading-indentation]
```

Trap: off by one

```
#include <stdio.h>

int main(void) {
    int i, count;

    count = 0;
    for (i = 0; i < 5; i++) count++;
    printf("i < 5  runs %d times\n", count);

    count = 0;
    for (i = 0; i <= 5; i++) count++;
    printf("i <= 5 runs %d times\n", count);
    return 0;
}
```

Output:

```
i < 5  runs 5 times
i <= 5 runs 6 times
```

Starting at 0 with `<` gives exactly n iterations. Starting at 1 with `<=` also gives exactly n. Mixing the two conventions gives n+1 or n-1, and that one extra iteration is what reads past the end of an array later in the program.

Also: do not modify the loop counter inside the body. It is legal and it is nearly always a bug.

The digit loop

This pattern appears in more campus coding tests than any other single technique, so it is worth having by heart.

```
#include <stdio.h>

int main(void) {
    int n = 4073;
    int sum = 0, count = 0, rev = 0;
    int work = n;

    while (work > 0) {
        int digit = work % 10;
        sum += digit;
        count++;
        rev = rev * 10 + digit;
        work /= 10;
    }
    printf("number      = %d\n", n);
    printf("digit sum   = %d\n", sum);
    printf("digit count = %d\n", count);
    printf("reversed    = %d\n", rev);
    return 0;
}
```

Output:

```
number      = 4073
digit sum   = 14
digit count = 4
reversed    = 3704
```

`% 10` takes the last digit off. `/ 10` removes it. Repeat until nothing is left. Note that the loop works on a copy called `work`, so the original `n` survives for the comparison at the end.



The zero trap. `while (work > 0)` never runs when the input is 0. The digit sum of 0 happens to come out right at 0, but the digit count comes out as 0 when it should be 1. Handle zero as a separate case before the loop.

---

13. Strings, the Basics

C has no string type. A string is an array of `char` with an invisible terminator character `'\0'` marking the end. Every string function in C finds the end by scanning for that terminator.

```
#include <stdio.h>
#include <string.h>

int main(void) {
    char name[20] = "RAGHAVENDRA";

    printf("Name        : %s\n", name);
    printf("Length      : %zu\n", strlen(name));
    printf("First char  : %c\n", name[0]);
    printf("Third char  : %c\n", name[2]);
    printf("Slot count  : %zu\n", sizeof(name));
    printf("Terminator  : %d\n", name[strlen(name)]);
    return 0;
}
```

Output:

```
Name        : RAGHAVENDRA
Length      : 11
First char  : R
Third char  : G
Slot count  : 20
Terminator  : 0
```

Four things here.

1. `#include <string.h>` is required for `strlen`. It is a different header from `stdio.h`.
2. `strlen` counts characters up to but not including the terminator, so 11. `sizeof` reports the declared size of the array, 20, regardless of what is in it. They are answering different questions.
3. **Indexing starts at 0.** `name[0]` is the first character, `name[2]` is the third, and the last character is at `name[strlen(name) - 1]`.
4. The terminator is the character with numeric value 0. The array must always have room for it, so `char name[20]` holds at most 19 usable characters.

Walking a string with a loop

```
#include <stdio.h>
#include <string.h>

int main(void) {
    char word[] = "HUBBALLI";
    int i;
    int vowels = 0;

    for (i = 0; i < (int)strlen(word); i++) {
        printf("position %d holds %c\n", i, word[i]);
        if (word[i] == 'A' || word[i] == 'E' || word[i] == 'I' ||
            word[i] == 'O' || word[i] == 'U') {
            vowels++;
        }
    }
    printf("vowel count = %d\n", vowels);
    return 0;
```



```
}
```

Output:

```
position 0 holds H
position 1 holds U
position 2 holds B
position 3 holds B
position 4 holds A
position 5 holds L
position 6 holds L
position 7 holds I
vowel count = 3
```

Writing `char word[] = "HUBBALLI"` without a size lets the compiler count for you and it allocates 9 slots, 8 characters plus the terminator.

Note that a single character is written with single quotes, `'A'`, and a string with double quotes, `"A"`. They are not interchangeable. `'A'` is one byte. `"A"` is two bytes, the letter and the terminator.

Reading a string, and its one limitation

```
#include <stdio.h>

int main(void) {
    char city[30];
    printf("Enter city: ");
    scanf("%29s", city);
    printf("Stored: [%s]\n", city);
    return 0;
}
```

Input:

```
NEW DELHI
```

Output:

```
Enter city: Stored: [NEW]
```

Two points, both important.

`%s` stops at the first space. It read `NEW` and abandoned `DELHI`. `scanf` with `%s` reads one word, never a line. Reading a full line with spaces uses a different function which is taught in class.

Note there is no `&` here. An array name used on its own already behaves as an address, so `scanf("%s", city)` is correct and `scanf("%s", &city)` is not what you want. This is the one place the ampersand rule does not apply, and the reason will make sense in the pointers block.

The `29` in `%29s` caps the read at 29 characters so a long input cannot overflow a 30 slot array. Get into the habit of writing it.

Everything else about strings, including the comparison, copying and searching functions, is taught in the classroom. You need only the above before day one.

---

14. The Trap Checklist

These nine cost more marks in campus coding tests than any gap in knowledge. Every one of them compiles cleanly and produces a wrong answer rather than an error message. Read this page again the night before your first coding test.

#	Trap	The fix
1	Integer division truncates. `7/2` is 3.	Cast before dividing, or use a decimal literal. `(float)a / b`
2	Silent integer overflow past 2147483647.	Use `long long` with `%lld` when values can be large.
3	`%d` with a `float`, or `%f` in `scanf` for a `double`.	Match the specifier to the type. `scanf` needs `%lf` for `double`.



#	Trap	The fix
4	= instead of == in a condition.	Read every condition back before moving on.
5	Ladder in the wrong order, loosest test first.	Tightest condition first. Trace one value through by hand.
6	Missing `break` in a `switch`.	One `break` per `case`, no exceptions.
7	Dangling `else` binding to the wrong `if`.	Braces on every branch, always.
8	Stray semicolon after `for (...)` or `if (...)`.	Look at the bracket, then look at the next character.
9	Leftover newline read by `%c`.	Write `scanf(" %c", &ch)` with the leading space.

Two more habits, not traps but close relatives. Initialise every variable at declaration. Compile with `gcc -Wall -Wextra` and fix every warning before you trust the output.

---

15. Practice Problems

Twenty problems, every one solvable with only what is in this handout. Nothing here needs arrays, functions or pointers.

Work in order. Problems 1 to 5 are warm-up and should take under five minutes each. Problems 6 to 15 are the working standard. Problems 16 to 20 sit at the difficulty of the first week of classroom problems, so treat them as the real test of readiness.

Sample input and expected output are given for each. Match the output format exactly, spacing included, because company coding platforms compare your output character by character.

Problem 1: Rectangle Area and Perimeter

Read the length and breadth of a rectangle as whole numbers. Print the area and the perimeter.

```
Input:  12 7
Output: Area = 84
        Perimeter = 38
```

Problem 2: Swap Two Numbers

Read two integers. Print them, swap them using a third variable, and print them again.

```
Input:  15 40
Output: Before: a = 15, b = 40
        After : a = 40, b = 15
```

Problem 3: Average of Three Marks

Read three integer marks. Print the total and the average to two decimal places. Watch the integer division trap.

```
Input:  78 85 91
Output: Sum = 254
        Average = 84.67
```

Problem 4: Seconds to Hours, Minutes and Seconds

Read a total number of seconds. Convert it to hours, minutes and seconds using only `/` and `%`.

```
Input:  9075
Output: 9075 seconds = 2 h 31 m 15 s
```

Problem 5: Character Report

Read a single character. Print the character, its numeric code, and the character that comes after it.

```
Input:  M
Output: Character : M
        ASCII code: 77
```



```
        Next char : N
```

Problem 6: Odd or Even, Positive or Negative

Read an integer. Print whether it is odd or even, then on the next line whether it is positive, negative or zero.

```
Input:  -34
Output: Even
        Negative
```

Problem 7: Largest of Three

Read three integers and print the largest. Do not use any library function.

```
Input:  45 92 67
Output: Largest = 92
```

Problem 8: Grade From Marks

Read marks out of 100. Print Grade S for 90 and above, Grade A for 80 to 89, Grade B for 70 to 79, Grade C for 40 to 69, and Fail below 40. If the input is below 0 or above 100, print INVALID and nothing else.

```
Input:  83
Output: Grade A
```

Problem 9: Simple Calculator

Read two decimal numbers, then read an operator character which is one of `+ - * /`. Print the result to two decimal places. Print DIVIDE BY ZERO if a division by zero is attempted, and INVALID OPERATOR for any other operator. Use `switch`. Remember the leading space in `scanf(" %c", &op)`.

```
Input:  45 8
        /
Output: Result = 5.62
```

Problem 10: Leap Year

Read a year. A leap year is divisible by 4 and not by 100, or divisible by 400. Print the verdict.

```
Input:  1900
Output: 1900 is not a leap year
```

Problem 11: Multiplication Table

Read a number and print its table from 1 to 10.

```
Input:  7
Output: 7 x 1 = 7
        7 x 2 = 14
        ... continuing to ...
        7 x 10 = 70
```

Problem 12: Odd and Even Tally

Read a count N, then read N integers. Print how many were even and their total, then how many were odd and their total.

```
Input:  6
        12 7 40 9 3 26
Output: Even count = 3, Even sum = 78
        Odd count  = 3, Odd sum  = 19
```

Problem 13: Sum and Factorial

Read a number N. Print the sum of the first N natural numbers and the factorial of N. Use `long long` for the factorial.

```
Input:  8
Output: Sum of first 8 numbers = 36
        Factorial of 8 = 40320
```

Problem 14: Reverse and Palindrome

Read an integer. Print it reversed, then print PALINDROME if the reversed value equals the original and NOT PALINDROME otherwise.

```
Input:  12321
```



```
Output: Reversed = 12321
        PALINDROME
```

Problem 15: Digit Count and Digit Sum

Read an integer. Print how many digits it has and the sum of those digits. Handle the input 0 correctly, and handle a negative input by working on its absolute value.

```
Input:  0
Output: Digit count = 1
        Digit sum   = 0
```

Problem 16: Right Aligned Star Triangle

Read N. Print a triangle of stars right aligned against the last column, so row 1 has one star preceded by N-1 spaces and row N has N stars and no spaces.

Input:

```
5
```

Output:

```
    *
   **
  ***
 ****
*****
```

Problem 17: Number Pyramid

Read N. Print a centred pyramid where row i counts up from 1 to i and back down to 1, with N-i leading spaces.

Input:

```
4
```

Output:

```
   1
  121
 12321
1234321
```

Problem 18: Prime Check and Prime List

Read N. Print whether N itself is prime, then print every prime from 2 up to N on one line separated by single spaces. Use `break` to stop checking a number as soon as a divisor is found.

```
Input:  20
Output: 20 is not prime
        Primes up to 20: 2 3 5 7 11 13 17 19
```

Problem 19: Character Tally in a Word

Read one word with no spaces, up to 49 characters. Print its length, then the counts of vowels, consonants, digits and any other characters. Treat upper and lower case as the same letter.

```
Input:  Bengaluru560001
Output: Length     = 15
        Vowels     = 4
        Consonants = 5
        Digits     = 6
        Others     = 0
```

Problem 20: Largest and Second Largest

Read a count N, then read N integers. Print the largest and the second largest. If every value is identical so that no distinct second largest exists, print NONE in its place. Solve this in a single pass, holding a running best and second best. Duplicates of the largest must not become the second largest.

```
Input:  7
        34 91 12 91 78 5 60
Output: Largest = 91
        Second largest = 78
```

16. Self-Assessment

Count a problem as solved only if it compiles with no warnings and produces exactly the expected output for the sample input, and also behaves sensibly on an input you invent yourself.

Solved	Where you stand	What to do
18 to 20	Well prepared. Your C has come back fully.	Re-read Section 14 and arrive ready to work at pace from the first hour.
14 to 17	Ready. The foundation is solid with a few gaps.	Identify which concept each failed problem belonged to and redo that section before day one.
10 to 13	Needs revision. The syntax is coming back but the traps are still catching you.	Work through Sections 7 to 13 again with the compiler open. Redo the diagnostic in Section 6. Re-attempt all twenty problems.
Below 10	Spend more time.	Start again at Section 3 and do not skip a single code example. Type every one of them yourself. Give this six to eight hours across the week before the program begins.

Two closing notes.

The scoring is for you, not for anyone else, so score it honestly. A copied solution that you did not reason through counts as unsolved.

If you are below 14, tell your trainer on day one. Knowing where the room stands lets the trainer pitch the first two sessions correctly, and the batches that speak up early are the batches that finish strongest.